Lab Practical 9

Design and develop a distributed Hotel booking application using Java RMI. A distributed hotel booking system consists of the hotel server and the client machines. The server manages hotel rooms booking information. A customer can invoke the following operations at his machine.
i) Book the room for the specific guest
ii) Cancel the booking of a guest.

HotelServiceInterface.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HotelServiceInterface extends Remote {
    String bookRoom(String guestName) throws RemoteException;
    String cancelBooking(String guestName) throws RemoteException;
    String viewBookings() throws RemoteException;
```

HotelServer.java

```
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;
import java.util.HashMap;

public class HotelServer extends UnicastRemoteObject implements HotelServiceInterface {

    private HashMap<String, String> bookings = new HashMap<>();

    HotelServer() throws RemoteException {
    }

    public String bookRoom(String guestName) throws RemoteException {
        if (bookings.containsKey(guestName)) {
            return "Room already booked for: " + guestName;
        }
        bookings.put(guestName, "Room Booked");
        return "Room successfully booked for: " + guestName;
    }

    public String cancelBooking(String guestName) throws RemoteException {
        if (!bookings.containsKey(guestName)) {
            return "No booking found for: " + guestName;
        }
```

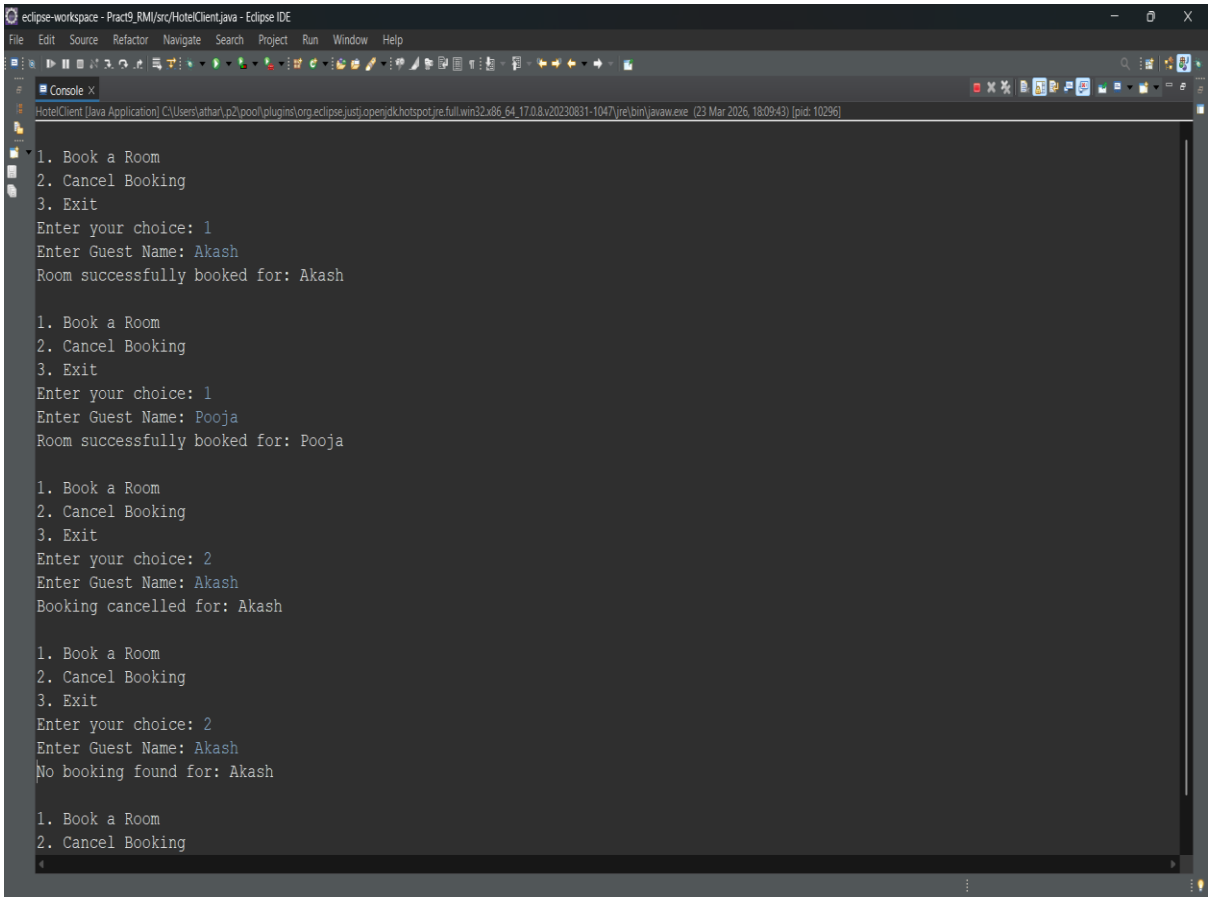
```
        bookings.remove(guestName);
        return "Booking cancelled for: " + guestName;
    }

    public String viewBookings() throws RemoteException {
        if (bookings.isEmpty()) {
            return "No current bookings.";
        }
        return "Current Bookings: " + bookings.keySet().toString();
    }

    public static void main(String[] args) {
        try {
            HotelServer server = new HotelServer();
            Registry registry = LocateRegistry.createRegistry(6000);
            registry.rebind("HotelService", server);
            System.out.println("Hotel Server is running and ready...");
        } catch (Exception e) {
            System.out.println("Server Error: " + e.getMessage());
        }
    }
}
```

HotelClient.java:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class HotelClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 6000);
            HotelServiceInterface hotel = (HotelServiceInterface) registry.lookup("HotelService");

            Scanner sc = new Scanner(System.in);
            int choice = 0;

            while (choice != 3) {
                System.out.println("\n1. Book a Room");
                System.out.println("2. Cancel Booking");
                System.out.println("3. Exit");
                System.out.print("Enter your choice: ");
                choice = sc.nextInt();
                sc.nextLine();

                switch (choice) {
                    case 1:
                        System.out.print("Enter Guest Name: ");
                        String guestBook = sc.nextLine();
                        System.out.println(hotel.bookRoom(guestBook));
                        break;
                    case 2:
                        System.out.print("Enter Guest Name: ");
                        String guestCancel = sc.nextLine();
```

```
                System.out.println(hotel.cancelBooking(guestCancel));
                break;
            case 3:
                System.out.println("Exiting...");
                break;
            default:
                System.out.println("Invalid choice!");
        }
    }
    sc.close();
  } catch (Exception e) {
    System.out.println("Client Error: " + e.getMessage());
  }
 }
}
```

Output:



Sign: _____.